EXPERIMENT - 5

AIM

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

OBJECTIVES

• To create a view that summarizes employee salary information department-wise.

• To create an employee hierarchy view using manager relationships.

• To test whether a view is updatable and understand the conditions affecting view updates.

• To implement a recursive CTE for displaying employee reporting chains.

• To verify the results using SELECT queries.

DATABASE USED

The queries use the CompanyDB database and the Employee–Department–Project schema from the reference experiment. The Employee table is extended with manager_id to support employee hierarchy and recursive reporting chains.

```
USE CompanyDB;
```

SCHEMA REFERENCE

The Department, Project and Employee tables remain connected through dept_id and project_id. The Employee table additionally contains manager_id, which references Employee(emp_id).

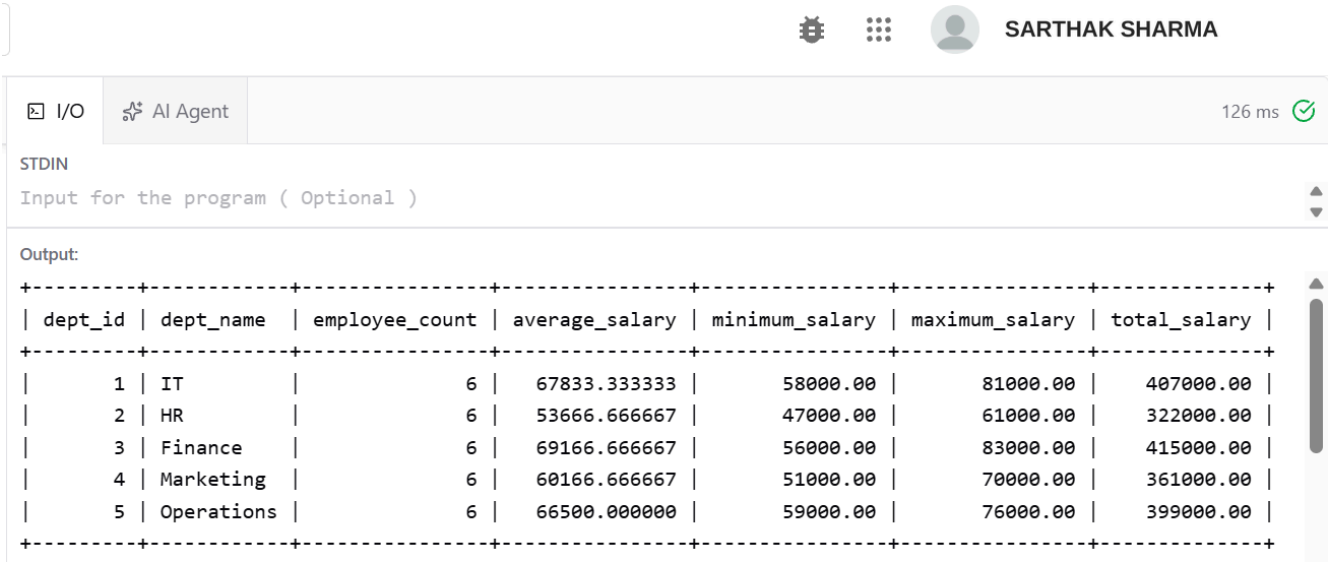
SOURCE CODE / SQL QUERIES

1. DEPARTMENT SALARY SUMMARY VIEW

Create a view showing employee count, average salary, minimum salary, maximum salary and total salary for each department.

```
CREATE OR REPLACE VIEW department_salary_summary AS
SELECT
    d.dept_id,
    d.dept_name,
    COUNT(e.emp_id) AS employee_count,
    AVG(e.salary) AS average_salary,
    MIN(e.salary) AS minimum_salary,
    MAX(e.salary) AS maximum_salary,
    SUM(e.salary) AS total_salary
FROM Department d
JOIN Employee e
    ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

Output



The view returns one summary row for each of the five departments.

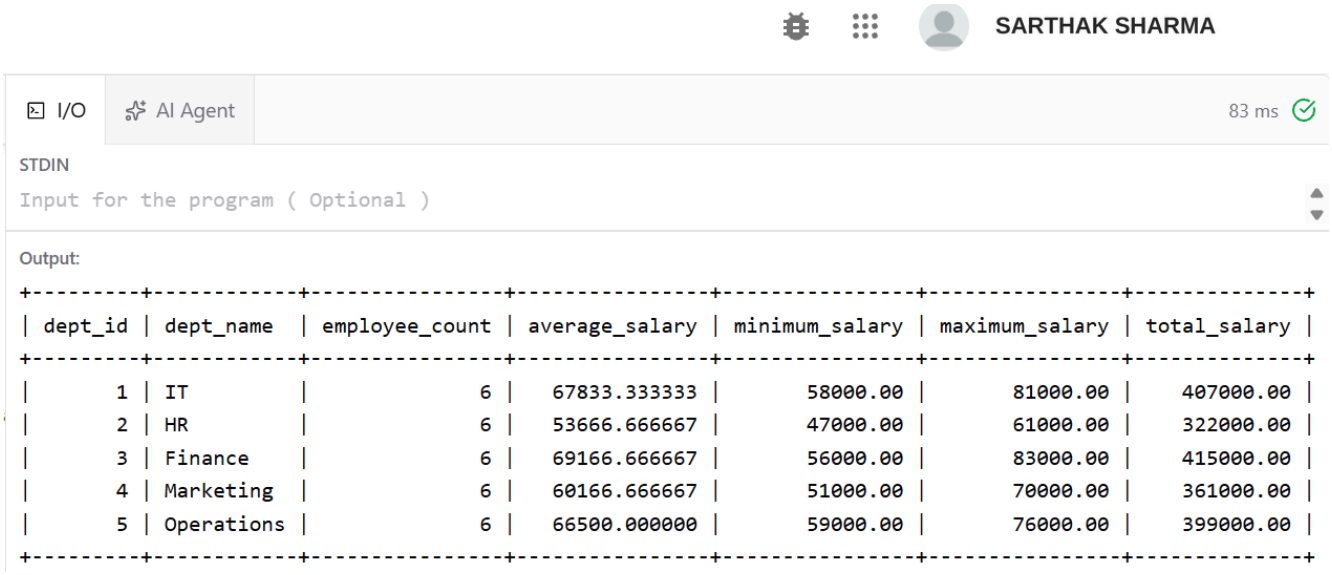
2. EMPLOYEE HIERARCHY VIEW

Create a view containing each employee and the name of the employee's immediate manager.

```
CREATE OR REPLACE VIEW employee_hierarchy AS
SELECT
    e.emp_id,
    e.emp_name,
    e.dept_id,
    d.dept_name,
    e.manager_id,
```

```
    m.emp_name AS manager_name,
    e.salary
FROM Employee e
JOIN Department d
    ON e.dept_id = d.dept_id
LEFT JOIN Employee m
    ON e.manager_id = m.emp_id;
```

Output



The hierarchy view shows the immediate reporting relationship. A department head has NULL in manager_id and manager_name.

3. TEST UPDATABILITY OF VIEWS

Test an update on a simple employee hierarchy view and then test an update on the grouped department salary summary view.

```
UPDATE employee_hierarchy
SET salary = salary + 1000
WHERE emp_id = 6;

SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id = 6;

UPDATE department_salary_summary
SET average_salary = average_salary + 1000
WHERE dept_id = 1;
```

Output

The first UPDATE is expected to fail because the employee_hierarchy view contains joins and does not directly expose a single base-table row for every selected column. The grouped

department_salary_summary view is also not directly updatable because it contains GROUP BY and aggregate functions. The base Employee table should be updated instead.

4. RECURSIVE CTE – REPORTING CHAIN

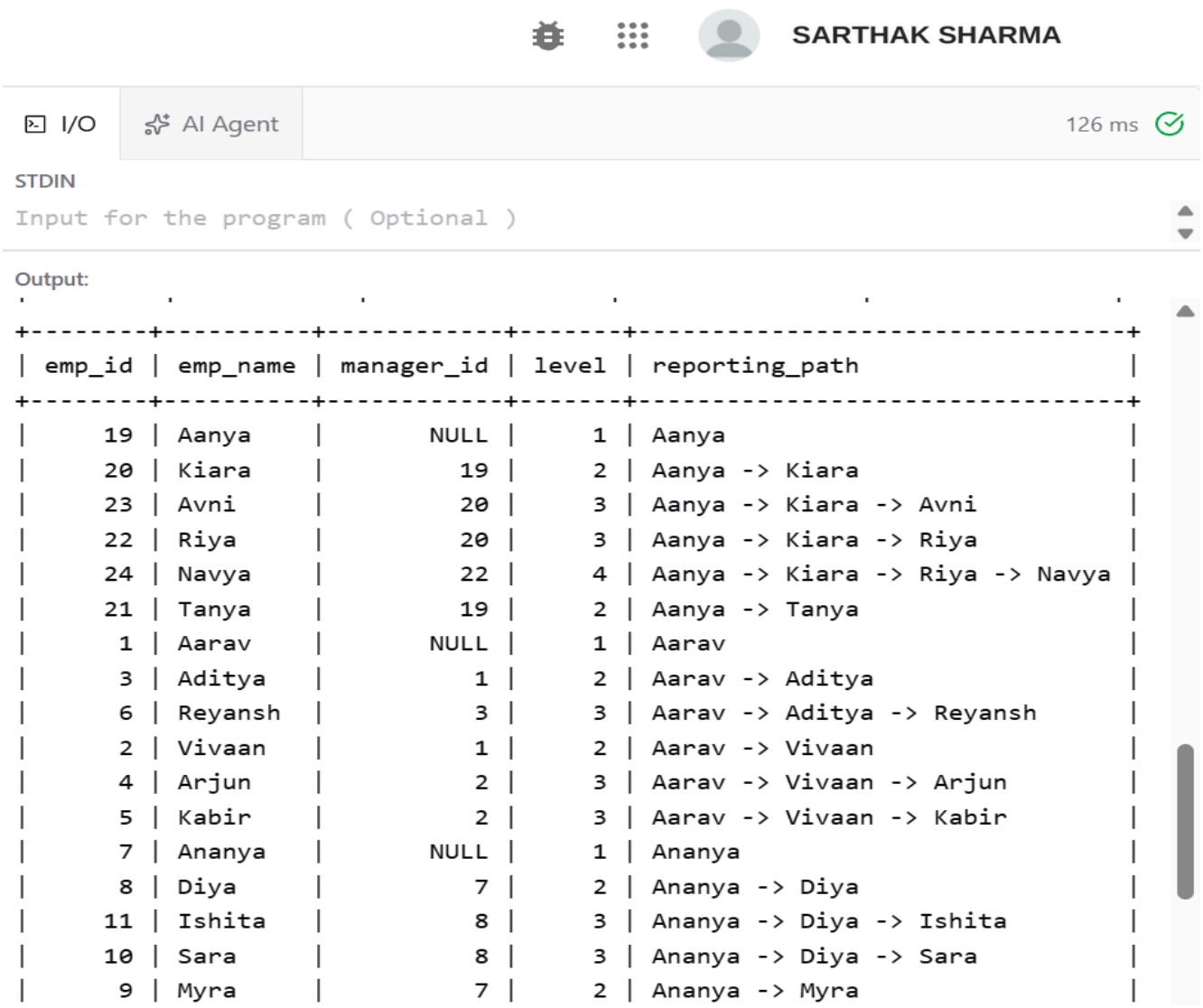
Display the reporting chain beginning with department heads and recursively follow manager relationships.

```
WITH RECURSIVE reporting_chain AS (
    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,
        1 AS level,
        CAST(e.emp_name AS CHAR(500)) AS reporting_path
    FROM Employee e
    WHERE e.manager_id IS NULL

    UNION ALL

    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,
        rc.level + 1,
        CONCAT(rc.reporting_path, ' -> ', e.emp_name)
    FROM Employee e
    JOIN reporting_chain rc
        ON e.manager_id = rc.emp_id
)
SELECT emp_id, emp_name, manager_id, level, reporting_path
FROM reporting_chain
ORDER BY reporting_path;
```

Output



emp_id	emp_name	manager_id	level	reporting_path
19	Aanya	NULL	1	Aanya
20	Kiara	19	2	Aanya -> Kiara
23	Avni	20	3	Aanya -> Kiara -> Avni
22	Riya	20	3	Aanya -> Kiara -> Riya
24	Navya	22	4	Aanya -> Kiara -> Riya -> Navya
21	Tanya	19	2	Aanya -> Tanya
1	Aarav	NULL	1	Aarav
3	Aditya	1	2	Aarav -> Aditya
6	Reyansh	3	3	Aarav -> Aditya -> Reyansh
2	Vivaan	1	2	Aarav -> Vivaan
4	Arjun	2	3	Aarav -> Vivaan -> Arjun
5	Kabir	2	3	Aarav -> Vivaan -> Kabir
7	Ananya	NULL	1	Ananya
8	Diya	7	2	Ananya -> Diya
11	Ishita	8	3	Ananya -> Diya -> Ishita
10	Sara	8	3	Ananya -> Diya -> Sara
9	Myra	7	2	Ananya -> Myra

RESULT

Views were created for department salary summaries and employee hierarchy. View update behavior was tested, and a recursive CTE was used successfully to display reporting chains.

CONCLUSION

SQL views provide reusable logical representations of data, while recursive CTEs can traverse self-referencing employee relationships to display hierarchical reporting structures.